THE CATALA SYNTAX

english

Literate programming

```
# Title                                   Heading
### Sub-subtitle
```catala       ```catala-metadata        Code / metadata block
```              ```
> Module Mdl   Module declaration
> Using Mdl as M   Module import+alias
> Include: foo.catala_en   File inclusion
```catala-test-cli                        Command-line interface test
$ catala interpret --scope Scope1
```
```

Literals and types

```
true                 false                boolean
65,536                                    integer
65536.262144         37%                  decimal
$1,234,567.89                             money
|2026-10-23|                              date
254 day      -4 month      1 year         duration
Case1 content 12        Case2             Enum1
Struct1 { -- fld1: 9 -- fld2: 7% }        Struct1
Present content $34     Absent            optional of money
[ 12; 24; 36 ]                            list of integer
[]                                        list of
                                            anything of type t
(|2012-02-03|, $30, 1%)                   (date,money,decimal)
f of x, y equals                          decimal depends on
  y * x / $12.0                             x content money,
                                            y content decimal
```

Operators and built-ins

```
not a          a and b                    Logical operators
a or b         # "or otherwise"
a xor b        # exclusive or
- a           a + b          a - b         Arithmetic
a * b          a / b
a = b          a < b          a <= b       Comparison
a != b         a > b          a >= b
decimal of 44                             Conversions
money of 23.15
round of $9.99                            Rounding
Date.get_month of ...                     Date parts
Date.first_day_of_month of ...
```

Metadata declaration

```
declaration structure Struct1:            Structure declaration
  data fld1 content integer
  data fld2 content decimal
declaration enumeration Enum1:            Enumeration declaration
  -- Case1 content integer
  -- Case2
## Documentation for Scope1                Documentation string
#[test]                                   Test scope annotation
declaration scope Scope1:                 Scope declaration
  internal var1 content integer           Scope variable
    state before                          State transitions
    state after
  internal var2 condition                 Condition
  sub1 scope Scope0                       Sub-scope declaration
  output var3 content ...                 Input-output qualifiers
  input var4 content ...
  input output var5 content ...
  context var6 content ...
  context output var7 content ...
  output sub2 scope Scope0
declaration const content decimal         Global definition
  equals 17.1
declaration square content decimal        Global function definition
  depends on x content decimal
  equals x * x
```

Expressions

```
let x equals 36 - 5 in ...                Local definition
match expr with pattern                   Pattern matching
-- Case1 content x : ...
-- Case2 : ...
-- anything : ...
#[error.message = "err"] impossible       Unreachable code
if ... then ... else ...                  Conditional
#[debug.print = "message"] expr           Debug annotation
expr with pattern Case1                   Pattern test
          content x and x >= 2              (optional: var. binding)
struc1 but replace { -- fld2: 8% }        Field replacement
struc1.fld2          tuple1.2             Field, tuple element,
sub1.var0                                   subscope variable
f of $44.50, 1/3                          Function call
output of Scope1 with                     Direct scope call
  { -- fld1: 9 -- fld2: 15% }
var1 state before                         Variable state access
assertion x > 0 in ...                    Local assertion
```

Scope definition

```
scope Scope1: ...                         Defining scope contents
scope Scope1                              Use-wide condition
  under condition var1 >= 2: ...
definition var1 equals ...                Unconditional def.
definition var1                           Conditional definition
  under condition ...
  consequence equals ...
rule var2                                 Rule (definition for conditions)
  under condition var1 >= 2
  consequence ·not· fulfilled
definition f of x, y equals ...           Function def. or rule
label lbl1 definition var1 ...            Labeled def. or rule
exception lbl1 definition var1 ...        Exception to label
exception definition var1 ...             Exception to implicit
definition var1 state before              State definition
  equals ...
assertion var1 > 0                        Scope-level assertion
date round down·up                        Date rounding mode
```

List operations

```
lst contains 3                            Presence test
exists x among lst such that x > 2        Existence test
for all x among lst we have x > 2         For all test
map each x among lst to x + 2             Mapping
list of x among lst such that x > 2       Filter
map each x among lst such that x > 2      Filter + map
  to x - 2
map each (x, y) among (lst1, lst2)        Multiple mapping
  to x + y
lst1 ++ lst2                              Merge
number of lst                             Count
maximum of lst                            Extremum search
  or if list empty then -1                  (optional default)
content of x among lst                    First element satisfying
  such that x.fld1 = ...                    the condition
content of x among lst                    Arg-extremum search
  such that x * x is minimum
  or if list empty then -1                  (optional default)
sort lst in in·decreasing order           Sorting
sort all x among lst                      Sorting on criteria
  in in·decreasing order of x.fld2
  and then -x.fld1 ...                      (optional sub-criteria)
combine all x among lst                   Fold (loop and accumulate)
  in acc initially 0
  with acc + x
```

LA SYNTAXE DE CATALA

français

Programmation littéraire

```
# Titre                                  En-têtes
### Sous-sous-titre
# Article 1 | JORFARTI000012345678        Référence au journal officiel
# Article 2 | LEGIARTI000012345678
# Décision 3 | CETATEXT000012345678
```catala        ```catala-metadata      Bloc de code / métadonnées
```               ```
> Module Mdl   Déclaration de module
> Usage de Mdl en tant que M   Import de module
> Inclusion: foo.catala_en   Inclusion textuelle
```catala-test-cli                       Test intégré
$ catala interpret --scope Chp1
```
```

Littéraux et types

```
vrai                  faux                booléen
65536                                     entier
65536,262144          37%                 décimal
1 234 567,89€                             argent
|2026-10-23|                             date
254 jour      -4 mois      1 an           durée
Cas1 contenu 12         Cas2              Énum1
Struct1 { -- chp1: 9 -- chp2: 7% }        Struct1
Présent contenu 34€     Absent            optionnel de argent
[ 12; 24; 36 ]                            liste de entier

[]                                        liste de
                                            n'importe quel
                                            de type t
(|2012-02-03|, 30€, 1%)                   (date,argent,décimal)

f de x, y égal à                          décimal dépend de
  y * x / 12,0€                             x contenu argent,
                                            y contenu décimal
```

Opérations

```
non a           a et b                    Opérateurs logiques
a ou b          # "ou à défaut"
a ou bien b     # ou exclusif
- a            a + b         a - b        Arithmétique
a * b          a / b
a = b          a < b         a <= b       Comparaisons
a != b         a > b         a >= b
décimal de 44                             Conversions
argent de 23,15
arrondi de 9,99€                          Arrondis
Date.accès_année de ...                   Éléments de dates
Date.premier_jour_du_mois de ...
```

Déclaration des métadonnées

```
déclaration structure Struct1:            Déclaration de structure
  donnée chp1 contenu entier
  donnée chp2 contenu décimal
déclaration énumération Énum1:            Déclaration d'énumération
  -- Cas1 contenu entier
  -- Cas2
## Documentation de Chp1                  Texte de documentation
#[test]                                   Annot. champ de test
déclaration champ d'application Chp1:     Déclaration de champ
  interne var1 contenu entier             Variable de champ
    état avant                            Transitions d'état
    état après
  interne var2 condition                  Condition
  sub1 champ d'application Chp0           s/s champ
  résultat var3 contenu ...               Qualificateurs d'entrée-sortie
  entrée var4 contenu ...
  entrée résultat var5 contenu ...
  contexte var6 contenu ...
  contexte résultat var7 contenu ...
  résultat sub2
    champ d'application Chp0
déclaration const contenu décimal         Définition globale
  égal à 17,1
déclaration carré contenu décimal         Définition de fonction globale
  dépend de x contenu décimal
  égal à x * x
```

Expressions

```
soit x égal à 36 - 5 dans ...             Définition locale
selon expr sous forme                     Filtrage par motif
-- Cas1 contenu x : ...
-- n'importe quel : ...
#[error.message = "err"] impossible       Calcul inaccessible
si ... alors ... sinon ...                Branchement
#[debug.print = "message"] expr           Annot. de débug
expr sous forme Cas1                      Test de filtrage (optionnel: test contenu)
          contenu x et x >= 2
struc1 mais en remplaçant                 Remplacement de champs
  { -- chp2: 8% }
struc1.chp2          tuple1.2             Champ, élément de n-uplet, s/s-variable
sub1.var0
f de 44,50€, 1/3                          Appel de fonction
résultat de Chp1                          Appel direct de champ d'application
  avec { -- chp1: 9 -- chp2: 15% }
var1 état avant                           Accès à un état
assertion x > 0 dans ...                  Assertion locale
```

Définition de champ d'application

```
champ d'application Chp1: ...             Définition des membres
champ d'application Chp1                  Avec condition générale
  sous condition var1 >= 2: ...
définition var1 égal à ...                Déf. inconditionnelle
définition var1                           Définition conditionnelle
  sous condition ...
  conséquence égal à ...
règle var2                                Règle (définition de condition)
  sous condition var1 >= 2
  conséquence ·non· rempli
définition f de x, y égal à ...           Déf./règle fonction
étiquette étq1 définition var1 ...        Déf./règle étiquetée
exception étq1 définition var1 ...        Exc. à déf. étiquetée
exception définition var1 ...             Exception à implicite
définition var1 état avant égal à ...     Définition d'états
assertion var1 > 0                        Assertion de champ
date arrondi inf·supérieur                Mode arrondi dates
```

Opérations sur les listes

```
lst contient 3                            Test de présence
existe x parmi lst tel que x > 2          Test d'existence
pour tout x parmi lst on a x > 2          Test pour tout
transforme chaque x parmi lst             Application un-à-un
  en x - 2
liste de x parmi lst tel que x > 2        Filtrage
transforme chaque x parmi lst             Filtrage + application
  tel que x > 2 en x - 2
transforme chaque (x, y)                  Application multiple
  parmi (lst1, lst2) en x + y
lst1 ++ lst2                              Réunion
nombre de lst                             Comptage
maximum de lst                            Recherche d'extremum (optionnel: défaut)
  ou si liste vide alors -1
contenu de x parmi lst                    Premier élément qui satisfait la condition
  tel que x.chp1 = 0
contenu de x parmi lst                    Extremum selon critère (optionnel: défaut)
  tel que x * x est minimum
  ou si liste vide alors -1
trie lst par ordre dé·croissant           Tri
trie tout x parmi lst                     Tri selon critère
  par ordre dé·croissant de x.chp2        (optionnel: sous-critère)
  puis -x.chp1 ...
combine tout x parmi lst                  Fold (boucle avec accumulation)
  dans acc initialement 0
  avec acc + x
```